

CS324

Lectures / Training

Last lecture, we talked about the **model architecture** for large language models (e.g., the Transformer). In this lecture, we will discuss how to train large language models.

- 1 Objective functions
- 2 Optimization algorithms

Objective functions

We will consider objective functions for the three types of language models:

- 1 Decoder-only (e.g., GPT-3): compute unidirectional contextual embeddings, generate one token at a time
- 2 Encoder-only (e.g., BERT): compute bidirectional contextual embeddings
- 3 Encoder-decoder (e.g., T5): encode input, decode output

We can use any model that maps token sequences into contextual embeddings (e.g., LSTMs, Transformers):

$$\phi : \mathcal{V}^L \rightarrow \mathbb{R}^{d \times L}.$$

$$[\text{the, mouse, ate, the, cheese}] \xRightarrow{\phi} \left[\begin{pmatrix} 1 \\ 0.1 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -0.1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \end{pmatrix} \right].$$

Decoder-only models

Recall that an autoregressive language model defines a conditional distribution:

$$p(x_i \mid x_{1:i-1}).$$

We define it as follows:

CS324

- Map $x_{1:i-1}$ to contextual embeddings $\phi(x_{1:i-1})$.
- Apply an embedding matrix $E \in \mathbb{R}^{V \times d}$ to obtain scores for each token $E\phi(x_{1:i-1})_{i-1}$.
- Exponentiate and normalize it to produce the distribution over x_i .

Succinctly:

$$p(x_{i+1} \mid x_{1:i}) = \text{softmax}(E\phi(x_{1:i})_i).$$

Maximum likelihood. Let θ be all the parameters of large language models.

Let $\mathcal{D}$ be the **training data** consisting of a set of sequences. We can then follow the maximum likelihood principle and define the following negative log-likelihood objective function:

$$\mathcal{O}(\theta) = \sum_{x_{1:L} \in \mathcal{D}} -\log p_{\theta}(x_{1:L}) = \sum_{x_{1:L} \in \mathcal{D}} \sum_{i=1}^L -\log p_{\theta}(x_i \mid x_{1:i-1}).$$

There's more to say about how to efficiently optimize this function, but that's all there is for the objective.

Encoder-only models

Unidirectional to bidirectional. A decoder-only model trained using maximum likelihood above also produces (unidirectional) contextual embeddings, but we can provide stronger bidirectional contextual embeddings given that we don't need to generate.

BERT. We will first present the **BERT** objective function, which contains two terms:

- 1 Masked language modeling
- 2 Next sentence prediction

Take the example sequence for natural language inference (predict entailment, contradiction, or neutral):

CS324

$$x_{1:L} = [[\text{CLS}], \text{all}, \text{animals}, \text{breathe}, [\text{SEP}], \text{cats}, \text{breathe}].$$

There are two special tokens:

- **[CLS]**: contains the embedding used to drive classification tasks
- **[SEP]**: used to tell the model where the first (e.g., premise) versus second sequence (e.g., hypothesis) are.

Using our notation from the previous lecture, the BERT model is defined as:

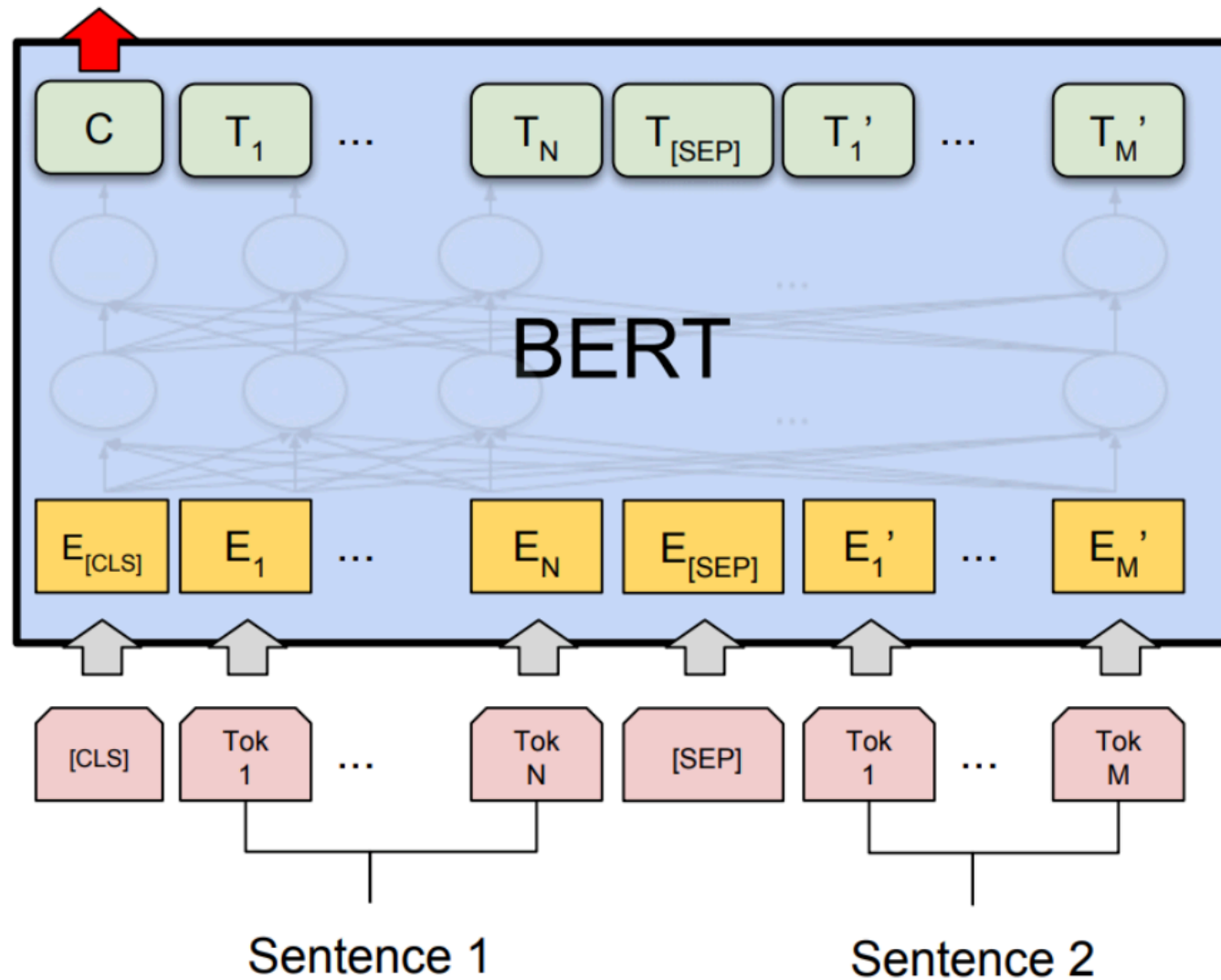
$$\text{BERT}(x_{1:L}) = \text{TransformerBlock}^{24}(\text{EmbedTokenWithPosition}(x_{1:L}) + \text{SentenceEmbedding}(x_{1:L})) \in \mathbb{R}^{d \times L}$$

where $\text{SentenceEmbedding}(x_{1:L})$ returns one of 2 vectors depending on the sequence:

- $e_A \in \mathbb{R}^d$ for tokens **left** of **[SEP]**, and
- $e_B \in \mathbb{R}^d$ for tokens **right** of **[SEP]**.

CS324

Class
Label



BERT-large has $n_{\text{heads}} = 16$ attention heads, and a $d_{\text{model}} = 1024$ dimensional model, resulting in 355M parameters.

CS324

Masked language modeling. The basic idea of the masked language model is to train on the prediction problem:

$$[\text{the}, [\text{MASK}], \text{ate}, [\text{MASK}], \text{cheese}] \Rightarrow [\text{the}, \text{mouse}, \text{ate}, \text{the}, \text{cheese}].$$

More more generally, we can think of this as similar to a denoising autoencoder where we map a noisy / incomplete version $\tilde{x}_{1:L}$ and try to reconstruct the original $x_{1:L}$.

$$\tilde{x}_{1:L} \Rightarrow x_{1:L}.$$

Model. We first define the model distribution that takes $\tilde{x}_{1:L}$ and predicts each token **independently** given the contextual embedding:

$$p(x_i \mid \tilde{x}_{1:L}) = \text{softmax}(E\phi(\tilde{x}_{1:L})_i).$$

Masking function. We define a (stochastic) noising function $A(\tilde{x}_{1:L} \mid x_{1:L})$ that:

$$\underbrace{x_{1:L}}_{\text{original}} \xRightarrow{A} \underbrace{\tilde{x}_{1:L}}_{\text{noised}}.$$

Here's how A is defined:

- 1 Let $I \subset \{1, \dots, L\}$ be a random 15% of the tokens positions.
- 2 For each $i \in I$:
 - With probability 0.8, set $\tilde{x}_i \leftarrow [\text{MASK}]$.
 - With probability 0.1, set $\tilde{x}_i \leftarrow x_i$.
 - With probability 0.1, set $\tilde{x}_i \leftarrow \text{random word from } \mathcal{V}$.

Reducing distribution shift. If we were to always replace chosen tokens in I with **[MASK]**, then:

CS324

- During training, every input BERT would only see sequences with a [MASK].
 - At test time, we would feed in sentences with no [MASK], resulting in a distribution shift.
- The heuristic fix is to replace with real words 20% of the time.

Next sentence prediction. Recall that BERT is trained on pairs of sentences concatenated. The goal of next sentence prediction is to predict whether the second sentence follows from the first or not.

[[CLS], the, mouse, ate, the, cheese, [SEP], it, was, full] $\Rightarrow 1$.

[[CLS], the, mouse, ate, the, cheese, [SEP], hello, world] $\Rightarrow 0$.

We will use the embedding of the [CLS] token to make this binary classification decision.

Dataset. Let $\mathcal{D}$ be a set of examples $(x_{1:L}, c)$ constructed as follows:

- Let A be a sentence from the corpus.
- With probability 0.5, let B be the next sentence.
- With probability 0.5, let B be a random sentence from the corpus.
- Let $x_{1:L} = [[\text{CLS}], A, [\text{SEP}], B]$.
- Let c denote whether B is the next sentence or not.

Objective. Then the BERT objective is:

$$\mathcal{O}(\theta) = \sum_{(x_{1:L}, c) \in \mathcal{D}} \underbrace{\mathbb{E}_{I, \tilde{x}_{1:L} \sim A(\cdot | x_{1:L}, I)} \left[\sum_{i \in I} -\log p_{\theta}(\tilde{x}_i | x_{1:L}) \right]}_{\text{masked language modeling}} + \underbrace{-\log p(c | \phi(x_{1:L})_1)}_{\text{next sentence prediction}}.$$

We will talk about training later, but a few quick notes about BERT:

CS324

- BERT (along with [ELMo](#) and [ULMFiT](#)) showed that one uniform architecture (Transformer) could be used for many multiple classification tasks.
- BERT really transformed the NLP community into a pre-training + fine-tuning mindset.
- BERT showed the importance of having deeply bidirectional contextual embeddings, although it's possible that model size and fine-tuning strategies make up for it ([p-tuning](#)).

[RoBERTa](#) makes the following changes to BERT:

- Removed the next sentence prediction objective (found it didn't help).
- Trained on more data (16GB text → 160GB text).
- Trained for longer. RoBERTa improved accuracy significantly over BERT on various benchmarks (e.g., on SQuAD 81.8 to 89.4).

Encoder-decoder models

Example task (table-to-text generation):

[name, :, Clowns, |, eatType, :, coffee, shop] ⇒ [Clowns, is, a, coffee, shop].

Recall that encoder-decoder models (e.g., BART, T5):

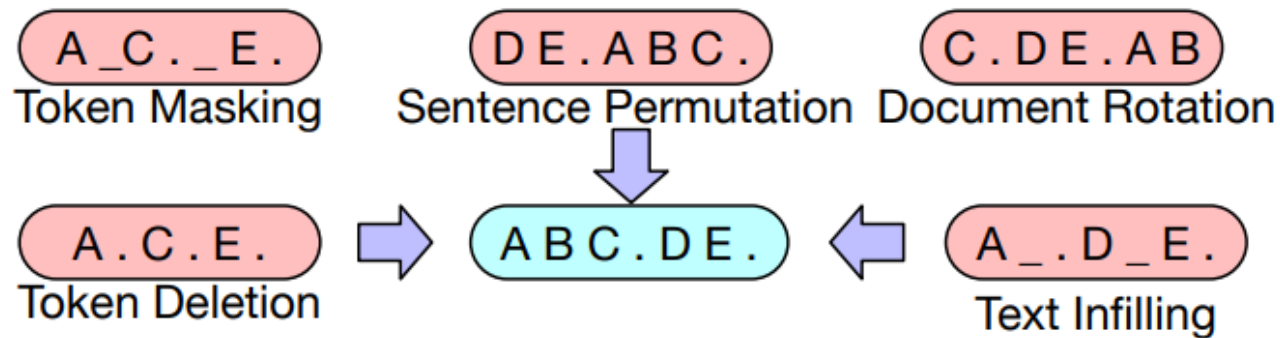
- Encode the input bidirectionally like BERT.
- Decode the output autoregressively like GPT-2.

BART (Bidirectional Auto-Regressive Transformers). BART ([Lewis et al. 2019](#)) is a Transformer-based encoder-decoder model.

- Same encoder architecture as RoBERTa (12 layers, hidden dimension 1024).
- Trained on same data as RoBERTa (160GB text).

CS324

BART considers the following transformations $A(\tilde{x}_{1:L} \mid x_{1:L})$:



Based on BERT-scaled experiments, they decided on the following transformations for the final model:

- Mask 30% of tokens in a document
- Permute all sentences

They demonstrated strong results on both classification and generation tasks using fine-tuning.

T5 (Text-to-Text Transfer Transformer).

T5 (Raffel et al., 2020) is another Transformer-based encoder-decoder model.

Tasks:

- Given a span of text, split at random point into input and output:

[the, mouse] $\Rightarrow$ [ate, the, cheese].

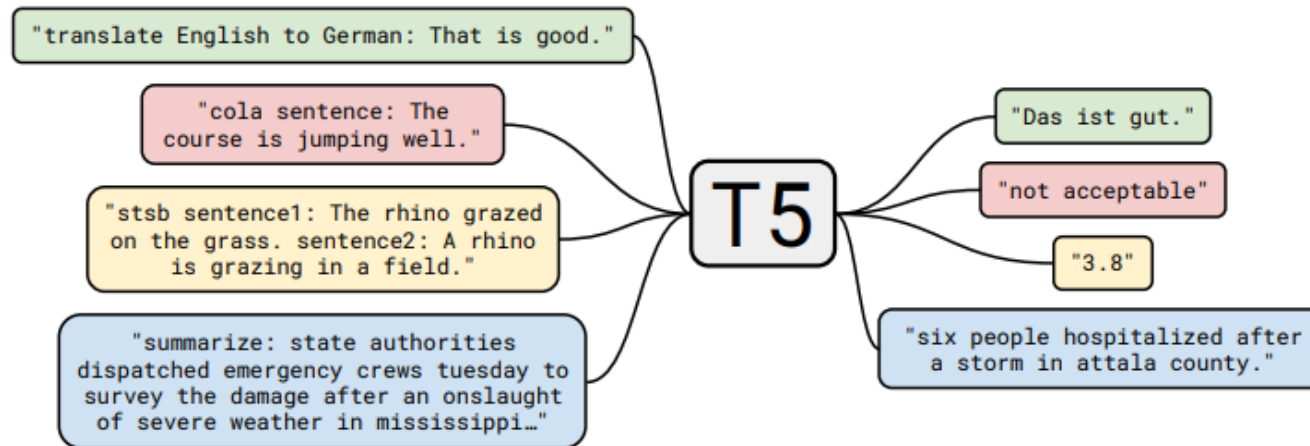
CS324

This paper experimented with many different unsupervised objectives:

Objective	Inputs	Targets
Prefix language modeling	Thank you for inviting	me to your party last week .
BERT-style Devlin et al. (2018)	Thank you <M> <M> me to your party apple week .	(original text)
Deshuffling	party me for your to . last fun you inviting week Thank	(original text)
MASS-style Song et al. (2019)	Thank you <M> <M> me to your party <M> week .	(original text)
I.i.d. noise, replace spans	Thank you <X> me to your party <Y> week .	<X> for inviting <Y> last <Z>
I.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last
Random spans	Thank you <X> to <Y> week .	<X> for inviting me <Y> your party last <Z>

and found that the “i.i.d. noise, replace spans” worked well (though many objectives were similar).

They also cast all classical NLP tasks in a uniform framework as “text-to-text” tasks:



Note the difference in approach to classification tasks:

- BERT used the embedding of the [CLS] token to predict.
- T5, GPT-2, GPT-3, etc. (models that can generate) cast the classification tasks in a **natural** language space.

Notes:

CS324

- The paper does a thorough **study** of many aspects of the entire pipeline (dataset, model size, training objective, etc.).
- Based on the insights, they trained a **11B parameter** model.

Optimization algorithms

Now we turn our attention to how to optimize the objective. For simplicity, let's take autogressive language modeling:

$$\mathcal{O}(\theta) = \sum_{x_{1:L} \in \mathcal{D}} -\log p_{\theta}(x_{1:L}).$$

Stochastic gradient descent (SGD). A first cut is just to do stochastic gradient descent with mini-batches:

- Initialize parameters θ_0 .
- Repeat:
 - Sample a mini-batch $B_t \subset \mathcal{D}$.
 - Perform a gradient step:

$$\theta_t \leftarrow \theta_{t-1} - \eta \frac{1}{|B_t|} \sum_{x_{1:L} \in B_t} \nabla_{\theta}(-\log p_{\theta}(x_{1:L})).$$

The key concerns in optimization are:

- 1 We want θ to converge **quickly** to a good solution.
- 2 We want the optimization to be numerically **stable**.
- 3 We want to be **memory efficient** (especially for large models). These are often at odds with each other (e.g., fast convergence and cutting down on memory by low-precision produces less stable training).

CS324

There are several levels that we can approach optimization:

- 1 Classic optimization: second-order methods, constrained optimization, etc.
- 2 Machine learning: stochastic methods, implicit regularization + early stopping
- 3 Deep learning: initialization, normalization (changes to the model architecture)
- 4 Large language models: stability issues, weird learning rates While some of the intuitions (e.g., second-order methods) are still useful, there are many other unique challenges that need to be overcome for large language model training to work. Unfortunately, much of this is fairly ad-hoc and poorly understood.

ADAM (adaptive moment estimation). ADAM incorporates two ideas:

- Use **momentum** (keep on moving in the same direction).
- Have an **adaptive** (different) step size for each dimension of θ (inspiration from second-order methods).
- Initialize parameters θ_0 .
- Initialize moments $m_0, v_0 \leftarrow 0$.
- Repeat:
 - Sample a mini-batch $B_t \subset \mathcal{D}$.
 - Update parameters as follows.

Updating parameters.

- Compute gradient:

$$g_t \leftarrow \frac{1}{|B_t|} \sum_{x_{1:L} \in B_t} \nabla_{\theta} (-\log p_{\theta}(x_{1:L})).$$

- Update first- and second-order moments:

CS324

$$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

- Do bias correction:

$$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$$

$$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$$

- Update parameters:

$$\theta_t \leftarrow \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon).$$

Memory. Using Adam increases the amount of storage from $2(\text{num-params})$ (from θ_t, g_t) to $4(\text{num-params})$ (from θ_t, g_t, m_t, v_t).

AdaFactor ([Shazeer & Stern, 2018](#)) was proposed as a way to reduce this memory footprint.

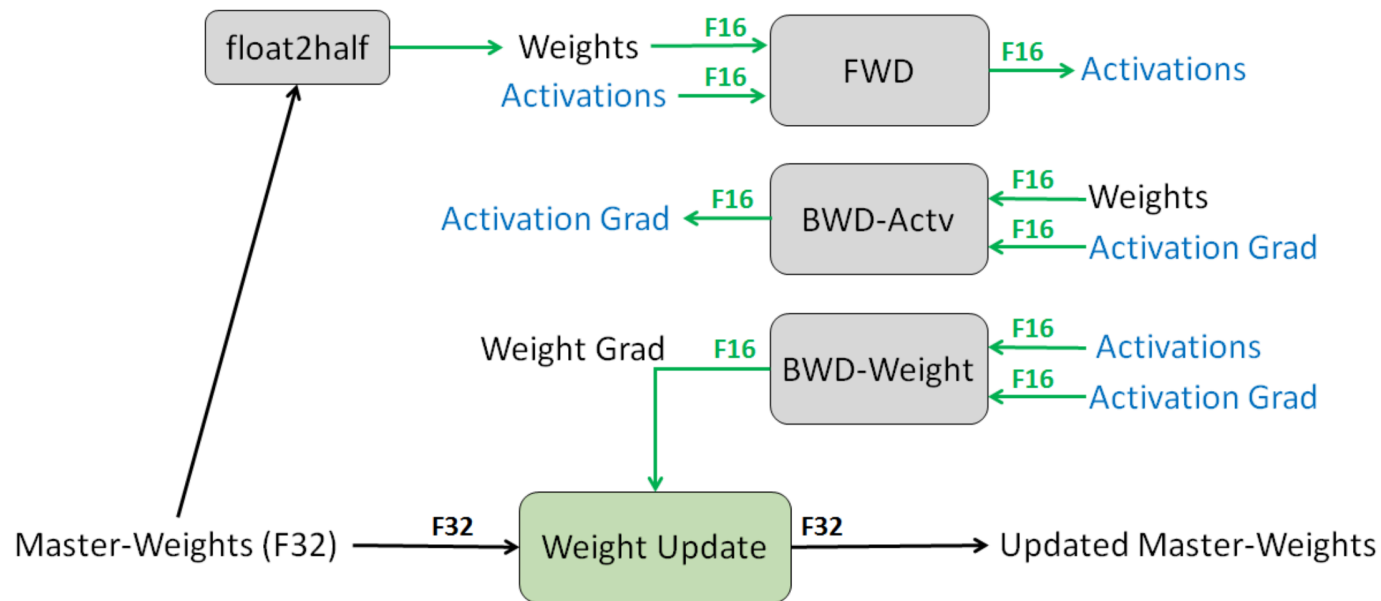
- Instead of storing the moments (m_t, v_t) of a $O(m \times n)$ matrix, store row and column sums ($O(m + n)$ memory) and reconstruct the matrix.
- Remove momentum.
- It was used to train T5.
- It can be difficult to get AdaFactor to train (see [Twitter thread](#) and [blog post](#)).

Mixed-precision training is another method for reducing memory ([Narang et al., 2018](#)).

- Default: FP32 (32-bit floating point).
- Option: FP16 (16-bit floating point), but the problem is that any value less than 2^{-24} becomes 0.
- Solution: store master weights in FP32 and do everything else in FP16.
- Loss scaling: scale up loss to avoid gradients with small magnitudes.

CS324

- Result: Halves the memory usage.



Learning rates.

- Normally, the learning rate **decreases** over time.
- For Transformers, we actually need to **increase** the learning rate (warmup).
- [Huang et al., 2020](#) show that a potential reason for this is to prevent vanishing gradients from layer normalization leads to instability in Adam optimizer.

Initialization.

- Given a matrix $W \in \mathbb{R}^{m \times n}$, the standard initialization (xavier initialization) is $W_{ij} \sim \mathcal{N}(0, 1/n)$, where n is the fan-in.
- GPT-2 and GPT-3 scale the weights by an additional $1/\sqrt{N}$, where N is the number of residual layers.
- T5 scales the attention matrices by an additional $1/\sqrt{d}$ ([code](#)).

CS324

For GPT-3:

- Adam parameters: $\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$.
- Batch size: 3.2 million tokens (~1500 sequences)
- Use gradient clipping ($g_t \leftarrow g_t / \min(1, \|g\|_2)$).
- Linear learning rate warmup (over first 375 million tokens).
- **Cosine learning rate** that goes down to 10% of value.
- Gradually increase the batch size.
- Weight decay 0.1.

Further reading

- **Mixed precision training**
- **Fixing Weight Decay Regularization in Adam.** *I. Loshchilov, F. Hutter.* 2017. Introduces **AdamW**.
- **ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators.** *Kevin Clark, Minh-Thang Luong, Quoc V. Le, Christopher D. Manning.* ICLR 2020.
- **DeBERTa: Decoding-enhanced BERT with Disentangled Attention.** *Pengcheng He, Xiaodong Liu, Jianfeng Gao, Weizhu Chen.* ICLR 2020.